Analyze the two GenAI programs. The analysis must include:

a. **Name of two GenAI programs**

I had used Claude Sonnet 5 and ChatGPT to create my two programs.

b. **How you accessed them (e.g., Google query, CoPilot on my computer)**

I accessed both GenAI programs by using Google and navigating to their respective websites. From their, I inputted my prompts.

c. **Prompt used. (You must use the same prompt for both GenAIs.)**

Implement a c++ program that uses objects to do the following. include lots of thorough commenting.

• The program will prioritize emails for a busy company CEO.

• You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.

• You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.

• You must create functions from scratch. Do not include pre-existing heap modules

The test file will begin

with the emails in the CEOs Inbox, followed by the commands the CEO will

use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Test File format:

Command	Description
EMAIL <sender category>, <subject line>, <date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

- Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

- Your program should display this:

```
There are 6 emails to read.

Next email:
        Sender: Boss
        Subject: Never Mind
        Date: 01-03-2025

Next email:
        Sender: Boss
        Subject: Important
        Date: 12-20-2024

There are 4 emails to read.
```

d. Generated code from both GenAIs.

Claude:

```
// ============================================================================
// ceo_email_queue.cpp
//
// Prioritizes emails for a busy CEO using a hand-built MaxHeap that serves as
// a priority queue.
//
// Design overview
// ---------------
// Email - a simple object holding one email's data plus the logic
// that decides whether it outranks another email.
// MaxHeap - a from-scratch, list-based (array-backed) binary max-heap
// of Email objects. No std::vector, std::priority_queue,
// or any other pre-existing heap/container module is used.
// EmailManager - reads the test file, interprets each command
// (EMAIL / NEXT / READ / COUNT) and drives the MaxHeap.
//
// Priority rules (from the assignment)
// ------------------------------------
// 1. Sender category:
// Boss -> read first
// Subordinate / Peer / ImportantPerson -> read next (all equal level)
// OtherPerson -> read last
// 2. Within the same priority level, the NEWEST email is read first.
// 3. (Extra) If two emails are identical in level AND date, the one that
// arrived first in the file is read first. This keeps output
// deterministic, because a heap by itself is not a "stable" structure.
//
// Build: g++ -std=c++11 -Wall -o ceo_email_queue ceo_email_queue.cpp
// Run: ./ceo_email_queue testfile.txt
// ============================================================================

#include <iostream> // std::cout, std::cerr
#include <fstream> // std::ifstream
#include <string> // std::string
#include <cstdlib> // std::atoi

using namespace std;



// ============================================================================
// class Email
// ----------------------------------------------------------------------------
// Represents a single email in the CEO's inbox.
// An Email knows how to rank itself against another Email, which is the only
// thing the MaxHeap needs in order to keep the most important email on top.
// ============================================================================
class Email {
private:
```

```
string senderCategory; // "Boss", "Subordinate", "Peer", ...
string subject; // subject line (may contain spaces, never commas)
string date; // original text, MM-DD-YYYY (kept for display)
int dateKey; // YYYYMMDD integer so dates compare numerically
int arrivalOrder; // order the email was added; used as a tie-breaker


// ------------------------------------------------------------------------
// Convert "MM-DD-YYYY" into the integer YYYYMMDD.
// Example: "01-03-2025" -> 20250103.
// A larger number always means a later (newer) date, so comparing two
// integers is enough to compare two dates. This also handles year
// rollovers correctly (01-03-2025 > 12-31-2024), which a plain string
// comparison of "MM-DD-YYYY" would get wrong.
// ------------------------------------------------------------------------
static int makeDateKey(const string& d) {
// Positions in "MM-DD-YYYY": MM = [0,1], DD = [3,4], YYYY = [6..9]
int month = atoi(d.substr(0, 2).c_str());
int day = atoi(d.substr(3, 2).c_str());
int year = atoi(d.substr(6, 4).c_str());
return year * 10000 + month * 100 + day;
}

public:
// Default constructor: needed because the heap's array is allocated with
// `new Email[capacity]`, which default-constructs every slot.
Email() : senderCategory(""), subject(""), date(""), dateKey(0), arrivalOrder(0) {}

// Full constructor used when an EMAIL command is parsed.
Email(const string& category, const string& subj, const string& dt, int order)
: senderCategory(category),
subject(subj),
date(dt),
dateKey(makeDateKey(dt)),
arrivalOrder(order) {}

// --- Simple accessors -----------------------------------------------
string getSenderCategory() const { return senderCategory; }
string getSubject() const { return subject; }
string getDate() const { return date; }

// ------------------------------------------------------------------------
// priorityLevel()
// Translate the sender category into a number; bigger = read sooner.
// Subordinate, Peer and ImportantPerson are all "read next", so they share
// the same level (and are therefore ordered by date among themselves).
// ------------------------------------------------------------------------
int priorityLevel() const {
if (senderCategory == "Boss") return 3; // read first
if (senderCategory == "Subordinate" ||
senderCategory == "Peer" ||
senderCategory == "ImportantPerson") return 2; // read next
return 1; // OtherPerson: last
```

```
}

// ------------------------------------------------------------------------
// hasHigherPriorityThan()
// Returns true if THIS email should be read before `other`.
// This single function defines the heap ordering.
// ------------------------------------------------------------------------
bool hasHigherPriorityThan(const Email& other) const {
// 1) Compare sender category level.
if (priorityLevel() != other.priorityLevel())
return priorityLevel() > other.priorityLevel();

// 2) Same level: the NEWER date (larger key) wins.
if (dateKey != other.dateKey)
return dateKey > other.dateKey;

// 3) Same level and same date: whoever arrived first wins.
return arrivalOrder < other.arrivalOrder;
}
};



// ============================================================================
// class MaxHeap
// ----------------------------------------------------------------------------
// A list-based (array-backed) binary max-heap of Email objects.
//
// The complete binary tree is stored level by level in a plain array:
//
// index: 0 1 2 3 4 5 ...
// tree: root L R LL LR RL ...
//
// For the element at index i:
// parent = (i - 1) / 2
// left child = 2 * i + 1
// right child = 2 * i + 2
//
// Heap property: every element has priority >= both of its children, so the
// highest-priority email is always at index 0.
//
// The array grows automatically (doubling) so there is no fixed inbox limit.
// ============================================================================
class MaxHeap {
private:
Email* data; // dynamically allocated array holding the heap
int count; // how many emails are currently stored
int capacity; // how many emails the array can hold before growing

// --- Index helpers ---------------------------------------------------
static int parentOf(int i) { return (i - 1) / 2; }
static int leftOf(int i) { return 2 * i + 1; }
static int rightOf(int i) { return 2 * i + 2; }
```

```
// Swap two Email objects in the array.
void swapEmails(int a, int b) {
Email temp = data[a];
data[a] = data[b];
data[b] = temp;
}

// ---------------------------------------------------------------------
// grow()
// Double the array size when it is full: allocate a bigger array, copy
// everything over, and free the old one.
// ---------------------------------------------------------------------
void grow() {
int newCapacity = capacity * 2;
Email* bigger = new Email[newCapacity];
for (int i = 0; i < count; i++)
bigger[i] = data[i];
delete[] data;
data = bigger;
capacity = newCapacity;
}

// ---------------------------------------------------------------------
// siftUp()
// Used after an insert. The new email starts at the bottom of the tree;
// keep swapping it with its parent while it outranks the parent.
// ---------------------------------------------------------------------
void siftUp(int i) {
while (i > 0) {
int p = parentOf(i);
if (data[i].hasHigherPriorityThan(data[p])) {
swapEmails(i, p); // child outranks parent -> move it up
i = p;
} else {
break; // heap property restored
}
}
}

// ---------------------------------------------------------------------
// siftDown()
// Used after removing the root. The last email is moved to the root and
// may be out of place; keep swapping it with its HIGHER-priority child
// until it is at least as important as both children.
// ---------------------------------------------------------------------
void siftDown(int i) {
while (true) {
int left = leftOf(i);
int right = rightOf(i);
int largest = i; // index of the highest-priority of {i, left, right}
```

```
if (left < count && data[left].hasHigherPriorityThan(data[largest]))
largest = left;
if (right < count && data[right].hasHigherPriorityThan(data[largest]))
largest = right;

if (largest == i)
break; // node is in the right place

swapEmails(i, largest);
i = largest; // continue from the child's position
}
}

public:
// Constructor: start with a small array; it will grow as needed.
MaxHeap() : data(new Email[8]), count(0), capacity(8) {}

// Destructor: release the dynamically allocated array.
~MaxHeap() { delete[] data; }

// Copying a heap is not needed here. Disabling it prevents accidental
// shallow copies that would double-delete the array.
MaxHeap(const MaxHeap&) = delete;
MaxHeap& operator=(const MaxHeap&) = delete;

// Number of emails currently waiting.
int size() const { return count; }
bool isEmpty() const { return count == 0; }

// ---------------------------------------------------------------------
// insert() - O(log n)
// Place the email in the first free slot (bottom of the tree), then sift
// it up to its proper position.
// ---------------------------------------------------------------------
void insert(const Email& e) {
if (count == capacity)
grow();
data[count] = e;
siftUp(count);
count++;
}

// ---------------------------------------------------------------------
// peekMax() - O(1)
// Look at the highest-priority email WITHOUT removing it (for NEXT).
// Caller must make sure the heap is not empty.
// ---------------------------------------------------------------------
const Email& peekMax() const { return data[0]; }

// ---------------------------------------------------------------------
// removeMax() - O(log n)
```

```
// Remove the highest-priority email (for READ):
// 1. Move the last email into the root slot.
// 2. Shrink the heap by one.
// 3. Sift the new root down to restore the heap property.
// Caller must make sure the heap is not empty.
// ---------------------------------------------------------------------------
void removeMax() {
data[0] = data[count - 1];
count--;
if (count > 0)
siftDown(0);
}
};



// ===========================================================================
// class EmailManager
// ---------------------------------------------------------------------------
// Owns the MaxHeap and interprets the commands found in the test file:
//
// EMAIL <sender category>,<subject line>,<date> add an email
// NEXT show the top email
// READ CEO handled the top email
// COUNT show unread count
// ===========================================================================
class EmailManager {
private:
MaxHeap inbox; // the priority queue of unread emails
int nextArrival; // running counter used as the arrival tie-breaker


// Remove leading/trailing spaces, tabs and carriage returns (so files with
// Windows "\r\n" line endings still parse correctly).
static string trim(const string& s) {
const string whitespace = " \t\r\n";
size_t start = s.find_first_not_of(whitespace);
if (start == string::npos)
return ""; // string was all whitespace
size_t end = s.find_last_not_of(whitespace);
return s.substr(start, end - start + 1);
}


// ---------------------------------------------------------------------------
// handleEmail()
// `args` is everything after "EMAIL ", i.e. "<category>,<subject>,<date>".
// The subject may contain spaces but never commas, so splitting on the
// first and last comma is safe.
// ---------------------------------------------------------------------------
void handleEmail(const string& args) {
size_t firstComma = args.find(',');
size_t lastComma = args.rfind(',');


// Malformed line: need two distinct commas. Skip it with a warning.
```

```
if (firstComma == string::npos || firstComma == lastComma) {
cerr << "Warning: skipping malformed EMAIL line: " << args << endl;
return;
}

string category = trim(args.substr(0, firstComma));
string subject = trim(args.substr(firstComma + 1, lastComma - firstComma - 1));
string date = trim(args.substr(lastComma + 1));

// Build the Email object and push it into the priority queue.
inbox.insert(Email(category, subject, date, nextArrival++));
}

// ---------------------------------------------------------------------
// handleNext() - display (but do not remove) the top-priority email.
// ---------------------------------------------------------------------
void handleNext() const {
if (inbox.isEmpty()) {
cout << "No emails to read." << endl << endl;
return;
}
const Email& e = inbox.peekMax();
cout << "Next email:" << endl;
cout << "\tSender: " << e.getSenderCategory() << endl;
cout << "\tSubject: " << e.getSubject() << endl;
cout << "\tDate: " << e.getDate() << endl;
cout << endl;
}

// ---------------------------------------------------------------------
// handleRead() - the CEO has dealt with the top email, so remove it.
// ---------------------------------------------------------------------
void handleRead() {
if (inbox.isEmpty()) {
cerr << "Warning: READ issued but there are no emails." << endl;
return;
}
inbox.removeMax();
}

// ---------------------------------------------------------------------
// handleCount() - display the number of unread emails.
// ---------------------------------------------------------------------
void handleCount() const {
cout << "There are " << inbox.size() << " emails to read." << endl << endl;
}

public:
EmailManager() : nextArrival(0) {}

// ---------------------------------------------------------------------
```

```
// processFile()
// Read the test file line by line and dispatch each command.
// Returns false if the file could not be opened.
// ------------------------------------------------------------------------
bool processFile(const string& filename) {
ifstream in(filename.c_str());
if (!in) {
cerr << "Error: could not open file '" << filename << "'" << endl;
return false;
}

string line;
while (getline(in, line)) {
line = trim(line);
if (line.empty())
continue; // ignore blank lines

if (line.compare(0, 6, "EMAIL ") == 0) {
handleEmail(line.substr(6)); // everything after "EMAIL "
} else if (line == "NEXT") {
handleNext();
} else if (line == "READ") {
handleRead();
} else if (line == "COUNT") {
handleCount();
} else {
cerr << "Warning: unrecognized command: " << line << endl;
}
}
return true;
}
};



// ============================================================================
// main
// Usage: ./ceo_email_queue <testfile>
// ============================================================================
int main(int argc, char* argv[]) {
if (argc < 2) {
cerr << "Usage: " << argv[0] << " <testfile>" << endl;
return 1;
}

EmailManager manager;
return manager.processFile(argv[1]) ? 0 : 1;
}
```

ChatGPT:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>

using namespace std;



/*
* ============================================================
* Email Class
* ============================================================
*
* This class represents one email in the CEO's inbox.
*
* Each email has:
* - A sender category
* - A subject
* - A date
* - A numeric priority for the sender category
* - A numeric value for the date
*
* The numeric values are used by the MaxHeap to determine which
* email should be at the top of the heap.
*/
class Email
{
private:
string sender;
string subject;
string date;

// Higher category priority means the email should be read first.
int senderPriority;
```

```
// The date converted into a number so dates can easily be compared.
// For example:
// 01-03-2025 becomes 20250103
int dateValue;

public:

/*
* Constructor
*
* Creates an Email object and calculates the values needed
* by the MaxHeap.
*/
Email(string senderCategory, string emailSubject, string emailDate)
{
sender = senderCategory;
subject = emailSubject;
date = emailDate;

senderPriority = calculateSenderPriority(senderCategory);
dateValue = convertDate(emailDate);
}



/*
* Converts the sender category into its priority.
*
* The assignment specifies:
*
* Boss - read first
* Subordinate - read next
* Peer - read next
* ImportantPerson - read next
* OtherPerson - read last
*
* Larger numbers represent higher priority because this is
* a MaxHeap.
*/
int calculateSenderPriority(string category)
{
if (category == "Boss")
{
return 5;
}
else if (category == "Subordinate")
{
return 4;
}
else if (category == "Peer")
{
return 3;
```

```
}
else if (category == "ImportantPerson")
{
return 2;
}
else
{
// OtherPerson
return 1;
}
}



/*
* Converts a date in MM-DD-YYYY format into YYYYMMDD.
*
* Example:
*
* 12-20-2024
*
* becomes:
*
* 20241220
*
* This makes it possible to compare two dates using normal
* integer comparisons.
*/
int convertDate(string emailDate)
{
int month = stoi(emailDate.substr(0, 2));
int day = stoi(emailDate.substr(3, 2));
int year = stoi(emailDate.substr(6, 4));

return year * 10000 + month * 100 + day;
}



/*
* Getter functions.
*
* These allow the MaxHeap to access the information it needs
* without directly accessing the private data members.
*/
string getSender()
{
return sender;
}

string getSubject()
{
return subject;
}
```

```
string getDate()
{
return date;
}

int getSenderPriority()
{
return senderPriority;
}

int getDateValue()
{
return dateValue;
}
};



/*
* ============================================================
* MaxHeap Class
* ============================================================
*
* This is our priority queue.
*
* We are NOT using C++'s priority_queue or any pre-existing
* heap implementation.
*
* Instead, we create the MaxHeap completely from scratch.
*
* The heap is represented using a dynamically allocated array.
*
* For an item at index i:
*
* Parent = (i - 1) / 2
*
* Left child = 2 * i + 1
*
* Right child = 2 * i + 2
*
* The largest/highest-priority email is always stored at index 0.
*/
class MaxHeap
{
private:
Email** heap;

// Number of emails currently stored.
int size;

// Maximum number of emails the current array can hold.
```

```
int capacity;



/*
* Determines whether one email has higher priority than another.
*
* Priority is determined in this order:
*
* 1. Sender category
* 2. Date
*
* This is important because the assignment's example shows:
*
* Boss, 01-03-2025
*
* should come before:
*
* Boss, 12-20-2024
*
* and both Boss emails come before a Subordinate email,
* even if the Subordinate email has a later date.
*/
bool higherPriority(Email* first, Email* second)
{
// First compare sender category.
if (first->getSenderPriority() != second->getSenderPriority())
{
return first->getSenderPriority() >
second->getSenderPriority();
}

// If the sender categories are the same, the newest
// email gets higher priority.
if (first->getDateValue() != second->getDateValue())
{
return first->getDateValue() >
second->getDateValue();
}

/*
* If both category and date are identical, neither email
* has a higher priority than the other.
*/
return false;
}



/*
* Doubles the size of the underlying list when it becomes full.
*
* This allows the MaxHeap to grow as more emails are added.
*/
```

```
void resize()
{
int newCapacity = capacity * 2;

Email** newHeap = new Email*[newCapacity];

// Copy the existing email pointers into the larger array.
for (int i = 0; i < size; i++)
{
newHeap[i] = heap[i];
}

// Delete the old array.
delete[] heap;

// Use the new array.
heap = newHeap;
capacity = newCapacity;
}



/*
* Restores the MaxHeap property by moving an email upward.
*
* This is called after inserting a new email.
*/
void heapifyUp(int index)
{
while (index > 0)
{
int parent = (index - 1) / 2;

// If the current email has higher priority than its
// parent, swap them.
if (higherPriority(heap[index], heap[parent]))
{
Email* temp = heap[index];
heap[index] = heap[parent];
heap[parent] = temp;

// Continue checking from the parent's position.
index = parent;
}
else
{
// The MaxHeap property has been restored.
break;
}
}
}
```

```
/*
* Restores the MaxHeap property by moving an email downward.
*
* This is called after removing the highest-priority email.
*/
void heapifyDown(int index)
{
while (true)
{
int leftChild = 2 * index + 1;
int rightChild = 2 * index + 2;

// Assume the current node has the highest priority.
int highest = index;



// Check whether the left child has higher priority.
if (leftChild < size &&
higherPriority(heap[leftChild], heap[highest]))
{
highest = leftChild;
}



// Check whether the right child has higher priority.
if (rightChild < size &&
higherPriority(heap[rightChild], heap[highest]))
{
highest = rightChild;
}



// If neither child has higher priority, the heap
// property is already correct.
if (highest == index)
{
break;
}



// Otherwise, swap the current email with the
// highest-priority child.
Email* temp = heap[index];
heap[index] = heap[highest];
heap[highest] = temp;

// Continue from the child's new position.
index = highest;
}
```

```
}



public:

/*
* Constructor
*
* Creates the initial list used by the MaxHeap.
*/
MaxHeap()
{
capacity = 10;
size = 0;

heap = new Email*[capacity];
}



/*
* Destructor
*
* Deletes the dynamically allocated array.
*
* The Email objects themselves are deleted when they are
* removed from the heap.
*/
~MaxHeap()
{
delete[] heap;
}



/*
* Inserts an email into the MaxHeap.
*
* Steps:
*
* 1. Resize if the array is full.
* 2. Put the email at the end of the heap.
* 3. Move it upward until the MaxHeap property is restored.
*/
void insert(Email* email)
{
if (size == capacity)
{
resize();
}

heap[size] = email;
```

```
heapifyUp(size);

size++;
}



/*
* Returns the highest-priority email without removing it.
*
* This is what the NEXT command needs.
*/
Email* peek()
{
if (size == 0)
{
return nullptr;
}

return heap[0];
}



/*
* Removes and returns the highest-priority email.
*
* The root of the MaxHeap is always the highest-priority email.
*
* Steps:
*
* 1. Save the root.
* 2. Move the last item to the root.
* 3. Decrease the size.
* 4. Heapify downward.
*/
Email* removeMax()
{
if (size == 0)
{
return nullptr;
}

Email* highestPriorityEmail = heap[0];

// Move the last email to the root.
heap[0] = heap[size - 1];

size--;

// Restore the MaxHeap property if emails remain.
```

```
if (size > 0)
{
heapifyDown(0);
}

return highestPriorityEmail;
}



/*
* Returns the current number of unread emails.
*/
int getSize()
{
return size;
}



/*
* Returns whether the heap is empty.
*/
bool isEmpty()
{
return size == 0;
}
};



/*
* ============================================================
* CEOInbox Class
* ============================================================
*
* This class handles the commands from the test file.
*
* It uses the MaxHeap as the priority queue for the CEO's emails.
*/
class CEOInbox
{
private:
MaxHeap emailQueue;



public:

/*
* Processes an EMAIL command.
*
* The input is expected to look like:
*
```

```
* EMAIL Boss,Important,12-20-2024
*/
void addEmail(string emailInformation)
{
string sender;
string subject;
string date;

/*
* stringstream allows us to split the information at
* commas.
*
* The subject is allowed to contain spaces, so we do not
* split based on spaces.
*/
stringstream input(emailInformation);

getline(input, sender, ',');
getline(input, subject, ',');
getline(input, date, ',');



// Create a new Email object.
Email* newEmail = new Email(sender, subject, date);



// Add the email to our MaxHeap.
emailQueue.insert(newEmail);
}



/*
* Handles the NEXT command.
*
* NEXT displays the highest-priority unread email but does
* NOT remove it from the heap.
*/
void nextEmail()
{
Email* email = emailQueue.peek();

if (email == nullptr)
{
cout << "There are no emails to read." << endl;
return;
}



cout << "Next email:" << endl;
```

```
cout << " Sender: " << email->getSender() << endl;
cout << " Subject: " << email->getSubject() << endl;
cout << " Date: " << email->getDate() << endl;

cout << endl;
}



/*
* Handles the READ command.
*
* READ means that the CEO has read and dealt with the
* highest-priority email.
*
* Therefore, we remove the maximum element from the heap.
*/
void readEmail()
{
Email* email = emailQueue.removeMax();

if (email != nullptr)
{
// The CEO has now dealt with this email, so the
// Email object is no longer needed.
delete email;
}
}



/*
* Handles the COUNT command.
*
* The heap's size is exactly the number of unread emails.
*/
void countEmails()
{
cout << "There are "
<< emailQueue.getSize()
<< " emails to read."
<< endl;

cout << endl;
}
};



/*
* ============================================================
* MAIN
* ============================================================
*
```

```
* The program can process commands either:
*
* 1. From a file supplied as a command-line argument:
*
* ./program test.txt
*
* or
*
* 2. From standard input:
*
* ./program < test.txt
*
* The second option is useful for many programming assignments.
*/
int main(int argc, char* argv[])
{
CEOInbox inbox;

/*
* This input stream will either refer to a file or to
* standard input.
*/
istream* input = &cin;

ifstream inputFile;



/*
* If a filename was provided as a command-line argument,
* open that file.
*/
if (argc > 1)
{
inputFile.open(argv[1]);

if (!inputFile)
{
cout << "Error: Could not open input file." << endl;
return 1;
}

input = &inputFile;
}



string line;



/*
* Read the test file one line at a time.
```

```
*/
while (getline(*input, line))
{
/*
* Ignore empty lines.
*/
if (line.empty())
{
continue;
}



/*
* Determine which command is being used.
*/
if (line.rfind("EMAIL ", 0) == 0)
{
/*
* Remove "EMAIL " from the beginning of the line.
*
* Example:
*
* EMAIL Boss,Important,12-20-2024
*
* becomes:
*
* Boss,Important,12-20-2024
*/
string emailInformation = line.substr(6);

inbox.addEmail(emailInformation);
}
else if (line == "NEXT")
{
inbox.nextEmail();
}
else if (line == "READ")
{
inbox.readEmail();
}
else if (line == "COUNT")
{
inbox.countEmails();
}
}



return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.

Correctness:

- The code works as expected and handles the intended user inputs well. When I tested both programs using the provided sample test file, both program produced an output identical to the desired display shown in the assignment.
- I tested a few edge cases like running the NEXT, READ, and COUNT commands when there are no emails to be read. Both ChatGPT and Claude were able to handle the edge cases and produced a very similar result, with the only difference being the message that is displayed in the in event that there are no emails to to be read. Claude would print out "Warning: READ issued but there are no emails." while ChatGPT would print "There are no emails to read." Regardless, both program produced the same correct output.

Execution Time:

- Both programs have a time complexity of O(nlog(n)) as they are both using a successive insertion algorithm where new emails are added to the bottom of the heap and sifts up.

Space Complexity:

- Because both programs implemented the priority queue using a heap, the space complexity to initialize a brand new heap is O(n) where each email takes up O(1) space. When the heap begins to grow full, a function gets called to double the size of the array used to implement the heap.

Maintainability:

- Both Claude and ChatGPT did a good job of seperating tasks from each other through the use of 3 main functions. And Email class, a MaxHeap class, and an eamil that simulates the CEO's inbox. Additionally, both AIs added lots of comments that help make functions more easily understaandable, however, there are not not comments on every line, just comments explaining blocks of code.

f. Select one to use as the basis for this assignment and justify why you picked it.

The GenAI program that I would choose is Claude's program. Both programs produced nearly identical results, however, Claude's program included more descriptive comments. Because there are more comments where every comment adds meaningful information, it would be easier in the future to maintain and read.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness:

- In ChatGPT's program, the heap contain pointers to dynamically allocated Email objects, and so when the program ends, emails that are still uread never got deallocated. This is a big memory management problem, and so I implrmented a function that actually gets rid of all the Email objects.
- In Claude's program, it always assumed some inputs would be standardized and predictable. While the assignments says that these assumptions should be true, I still impoemented a way to catch wierd behaviors like a typo in the catory and an impossible date.

Execution time:

- In ChatGPT's implementation of the calculateSenderPriority function, a string was always being compared to a category. I improved the comparison function by just using the precomputed integer value.
- In Claude's program, it was using the priorityLevel and hasHigherPriorityThan functions to compare a string literals multiple times. I changed this so that now, the priority is computed only once in the constructor and stored as an int so that every following comparison does not need to compare using strings, a slower task.

Space:

- In ChatGPT's program, there was no way to shrink the size of the array. This is especially important because there might be a scenario where the heap can grow to over 100 values, but the array is not using the full capacity so the program is wasting memory. I implemented a way to reduce the size.
- In Claude's program, the array could grow increasingly large, but if the CEO reads all emails, the memory will follow the historical peak rather than reflect the current

size of the array. I added functions that would detects when the heap was a quarter full and then would just halve the total size to reduce the space needed.

Maintainability:

- While Claude and ChatGPT created programs with good commenting already, I decided to add more and clarify anything that might be unfamiliar.